🎨 Flutter CustomClipper and CustomPaint Documentation

Overview

Flutter provides powerful tools to create custom UI shapes and graphics:

- **CustomClipper<Path>** → Cuts widgets into custom shapes
- **CustomPaint / CustomPainter** → Draws custom graphics using canvas

These tools allow you to create:

- Ticket shapes
- Wave containers
- Custom cards
- Graphs and charts
- Unique UI components

This documentation explains both concepts using a real production example.

Understanding CustomClipper

What is CustomClipper?

`CustomClipper<Path>` is used to define a custom shape to clip a widget.

It works with:

```
ClipPath(
  clipper: YourClipper(),
  child: YourWidget(),
)
```

🧩 Example: Small Edge Circle Clipper

This clipper creates:

• Rounded rectangle container
• Left circular cutout
• Right circular cutout

Like a ticket shape.

Full Source Code — Custom Clipper

```dart
import 'package:flutter/material.dart';

/// CustomClipper for small edge circular cutouts
class SmallEdgeCircleClipper extends CustomClipper<Path> {
  final double borderRadius;

  SmallEdgeCircleClipper({this.borderRadius = 16});

  @override
  Path getClip(Size size) {

    /// Step 1: Create base rounded rectangle
    Path rectPath = Path()
      ..addRRect(
        RRect.fromRectAndRadius(
          Rect.fromLTWH(0, 0, size.width, size.height),
          Radius.circular(borderRadius),
        ),
      );

    /// Step 2: Define circle radius
    double radius = size.height * 0.2;

    /// Step 3: Left circle
    Path leftCircle = Path()
      ..addOval(
        Rect.fromCircle(
          center: Offset(0, size.height / 1.5),
          radius: radius,
        ),
      );

    /// Step 4: Right circle
    Path rightCircle = Path()
      ..addOval(
        Rect.fromCircle(
```

```
          center: Offset(size.width, size.height / 2),
          radius: radius,
        ),
      );

    /// Step 5: Subtract circles from rectangle
    Path finalPath = Path.combine(
      PathOperation.difference,
      rectPath,
      leftCircle,
    );

    finalPath = Path.combine(
      PathOperation.difference,
      finalPath,
      rightCircle,
    );

    return finalPath;
  }

  @override
  bool shouldReclip(CustomClipper<Path> oldClipper) {
    return false;
  }
}
```

Container Widget Using Clipper

```
class SmallEdgeCircleContainer extends StatelessWidget {

  final double width;
  final double height;
  final Color color;
  final Widget? child;

  const SmallEdgeCircleContainer({
    super.key,
    required this.width,
    required this.height,
    this.color = Colors.blue,
    this.child,
  });
```

```
  @override
  Widget build(BuildContext context) {
    return ClipPath(

      clipper: SmallEdgeCircleClipper(
        borderRadius: 16,
      ),

      child: Container(
        width: width,
        height: height,
        color: color,

        child: child != null
            ? Center(
                child: Padding(
                  padding: const EdgeInsets.symmetric(horizontal: 16),
                  child: child,
                ),
              )
            : null,
      ),
    );
  }
}
```

How to Use

```
SmallEdgeCircleContainer(
  width: 300,
  height: 120,
  color: Colors.blue,

  child: Text(
    "Custom Clipped Container",
    style: TextStyle(
      color: Colors.white,
      fontSize: 18,
      fontWeight: FontWeight.bold,
    ),
  ),
)
```

🎨 Understanding CustomPaint

CustomPaint is used to draw graphics using canvas.

Structure:

```
CustomPaint(
  painter: MyPainter(),
  child: Widget(),
)
```

🧩 Example: Custom Painter

```
class CirclePainter extends CustomPainter {

  @override
  void paint(Canvas canvas, Size size) {

    final paint = Paint()
      ..color = Colors.blue
      ..style = PaintingStyle.fill;

    canvas.drawCircle(
      Offset(size.width / 2, size.height / 2),
      50,
      paint,
    );
  }

  @override
  bool shouldRepaint(CustomPainter oldDelegate) {
    return false;
  }
}
```

Using CustomPaint

```
CustomPaint(
  size: Size(200, 200),
  painter: CirclePainter(),
)
```

🎯 Real World Use Cases

Use CustomClipper for:

• Ticket UI
• Coupon cards
• Wave containers
• Custom cards

Use CustomPaint for:

• Charts
• Background design
• Animations
• Custom graphics

Performance Tips

Always return false if no repaint needed:

```
bool shouldRepaint(CustomPainter oldDelegate) => false;
```

```
bool shouldReclip(CustomClipper<Path> oldClipper) => false;
```

🧨 Complete Example

```
Scaffold(
  body: Center(
```

```
    child: SmallEdgeCircleContainer(
      width: 300,
      height: 120,
      color: Colors.blue,
      child: Text("Ticket Shape"),
    ),
  ),
)
```

Summary

CustomClipper → Cuts widgets
CustomPainter → Draws graphics
ClipPath → Applies clipper
CustomPaint → Applies painter

🎉 You are now ready to build custom shapes in Flutter.

If you want, I can also export this as:

• PDF
• DOCX
• Markdown

for download.